

AI Traffic Analyzer - Complete Project Guide

1. Project Overview

AI Traffic Analyzer is an AI-powered web application that analyzes uploaded traffic videos or live camera feeds. The system detects vehicles using a pretrained YOLO model, tracks them across frames, counts them, classifies them, estimates traffic density, and presents analytics through a modern dashboard. The project is intended as a professional college mini project that demonstrates Computer Vision, AI, Web Development, and Data Visualization.

2. Objectives

- Detect vehicles in real time.
- Count vehicles without double counting.
- Classify vehicles (car, bus, truck, bike, bicycle).
- Predict traffic density (Low/Medium/High).
- Display analytics dashboards.
- Generate downloadable PDF reports.

3. Technology Stack

Frontend: React + Tailwind CSS + Chart.js Backend: Flask (or FastAPI) AI: Python, OpenCV, Ultralytics YOLOv8/YOLOv11, ByteTrack/DeepSORT Database: SQLite (or MongoDB) Deployment: Local machine or Render/Vercel (frontend).

4. Project Architecture

Video/Webcam -> YOLO Detection -> Object Tracker -> Vehicle Counter -> Traffic Analyzer -> Backend API -> React Dashboard -> Reports & Charts

5. Core Features

1. Upload Video 2. Live Webcam Support 3. Vehicle Detection 4. Vehicle Classification 5. Vehicle Tracking 6. Vehicle Counting 7. Lane-wise Counting 8. Traffic Density Prediction 9. Smart Signal Recommendation 10. Interactive Dashboard 11. Charts (vehicle trends & distribution) 12. Heatmap of vehicle movement 13. Export PDF Report

6. Advanced Features (Bonus)

- Emergency vehicle detection
- Accident detection
- Number plate detection (optional)
- Speed estimation
- Historical analytics
- Dark mode

7. Folder Structure

frontend/ backend/ models/ uploads/ outputs/ reports/ database/ README.md

8. Development Roadmap

Phase 1: Create React frontend. Phase 2: Build Flask backend APIs. Phase 3: Integrate YOLO for vehicle detection. Phase 4: Add object tracking and counting. Phase 5: Build dashboard and charts. Phase 6: Add report generation. Phase 7: Testing and optimization. Phase 8: Prepare documentation and presentation.

9. Team Responsibilities

Member 1: React UI & Dashboard Member 2: Backend APIs & Database Member 3: AI Model, OpenCV, Tracking Member 4: Reports, Documentation, Testing, PPT

10. Deliverables

• Source code • README • Working demo • Presentation • Project report • Sample traffic videos • Final PDF analytics reports

11. Future Scope

Smart city integration, cloud deployment, adaptive traffic lights, edge AI cameras, parking occupancy analytics, and integration with government traffic systems.

12. Suggested Timeline (3 Weeks)

Week 1: Frontend + Backend + YOLO setup Week 2: Tracking, counting, dashboard Week 3: Reports, testing, documentation, presentation

13. Evaluation Highlights

Demonstrate real-time detection, accurate counting, clean UI, meaningful analytics, professional documentation, and explain the AI pipeline clearly.